Evaluación comparativa de ejecución de código MCP vs llamadas directas en sistemas multi-agente

2026

Resumen

Resumen pendiente de redacción.

Palabras clave: MCP, ejecución de código, sistemas multi-agente, evaluación comparativa, consumo de tokens.

Abstract. *Abstract pending.*

Keywords: MCP, code execution, multi-agent systems, comparative evaluation, token consumption.

Índice

1. Introducción **6**
1.1. Contexto . . . 6
1.2. Planteamiento del Problema . . . 7
1.3. Motivación . . . 9
1.3.1. Impacto económico . . . 9
1.3.2. Ausencia de guías para desarrolladores . . . 10
1.3.3. Crecimiento acelerado del ecosistema MCP . . . 10
1.3.4. Vacío en la literatura académica . . . 10
1.4. Objetivos . . . 11
1.4.1. Objetivo General . . . 11
1.4.2. Objetivos Específicos . . . 11
1.5. Hipótesis . . . 11
1.6. Alcance y Limitaciones . . . 11

2. Revisión de Literatura **12**
2.1. Protocolo MCP (Model Context Protocol) . . . 12
2.1.1. Origen y motivación . . . 12
2.1.2. Arquitectura cliente-servidor . . . 12
2.1.3. Capas del protocolo . . . 13
2.1.4. Flujo de interacción . . . 14
2.1.5. Adopción del ecosistema . . . 15
2.2. Ejecución de Código en LLMs . . . 15
2.2.1. El patrón de Anthropic: herramientas como código . . . 15
2.2.2. AWS Strands: code interpreter como herramienta . . . 17
2.2.3. E2B: microVMs para ejecución aislada . . . 17
2.2.4. Cloudflare Workers: WebAssembly como sandbox . . . 18
2.2.5. Comparación de enfoques . . . 18
2.3. Sistemas Multi-Agente . . . 19
2.3.1. Topologías de orquestación . . . 19
2.3.2. Frameworks existentes . . . 20
2.3.3. Patrones de comunicación . . . 22
2.4. Benchmarks Existentes . . . 23
2.4.1. Benchmarks de tool-use . . . 23
2.4.2. Benchmarks de agentes . . . 24

2.4.3. El vacío identificado 25
2.5. Sandboxing y Seguridad 25
2.5.1. Modelo de amenazas 26
2.5.2. Estrategias de mitigación en Docker 27
2.5.3. Alternativas de aislamiento 28
2.5.4. Implicaciones para la investigación 29

3. Marco Teórico **30**
3.1. Arquitectura MCP 30
3.2. Mecánica de Tool Calling 30
3.3. Ejecución de Código en Sandbox 30
3.4. Topologías Multi-Agente 30
3.5. Economía de Tokens 30

4. Diseño Experimental **31**
4.1. Matriz Factorial 2×2 31
4.2. Suite de Tareas 31
4.3. Servidores MCP Utilizados 31
4.4. Métricas 31
4.5. Variables de Control 31

5. Metodología **32**
5.1. Implementación del Framework 32
5.2. Sandbox Docker 32
5.3. Servidores MCP 32
5.4. Runner de Experimentos 32
5.5. Sistema de Evaluación 32

6. Resultados **33**
6.1. Resultados por Arquitectura 33
6.2. Análisis de Tokens 33
6.3. Análisis de Latencia 33
6.4. Análisis de Precisión 33
6.5. Análisis Estadístico 33

7. Modelo de Decisión **34**
7.1. Feature Engineering 34
7.2. Entrenamiento del Clasificador 34

7.3. Validación 34
7.4. Árbol de Decisión Interpretable 34

8. Discusión **35**
8.1. Interpretación de Resultados 35
8.2. Comparación con Literatura 35
8.3. Análisis de Modos de Fallo 35
8.4. Implicaciones Prácticas 35

9. Conclusiones **36**
9.1. Resumen de Hallazgos 36
9.2. Contribuciones 36
9.3. Trabajo Futuro 36

A. Apéndice **39**
A.1. Código Fuente de Servidores MCP 39
A.2. Tablas Completas de Resultados 39
A.3. Esquemas de Servidores MCP 39
A.4. Prompts Utilizados 39

1. Introducción

1.1. Contexto

Los modelos de lenguaje de gran escala (LLMs, por sus siglas en inglés) han alcanzado un nivel de capacidad sin precedentes en los últimos años. Modelos como GPT-5 (OpenAI, 2025), Claude Opus 4.6 (Anthropic, 2025) y Gemini 3 (Google DeepMind, 2025) demuestran desempeño competitivo en razonamiento matemático, generación de código, análisis de texto y resolución de problemas complejos. En múltiples escalas académicos y profesionales, estos sistemas igualan o superan el rendimiento humano en tareas que hace pocos años se consideraban fuera del alcance de la inteligencia artificial.

Sin embargo, conforme las capacidades fundamentales de los LLMs se consolidan, la frontera de investigación y desarrollo se ha desplazado hacia un problema distinto: la **gestión eficiente del contexto**. La ventana de contexto es el espacio limitado de tokens que un modelo puede procesar en una sola interacción, este se ha convertido en el recurso más valioso y escaso en aplicaciones para LLM's. Cada token consumido representa un costo económico directo, una contribución a la latencia de respuesta y una porción del espacio disponible para información relevante. En este sentido, el desafío ya no es si un LLM *puede* resolver una tarea, sino *cuán eficientemente* se puede minimizar el gasto al resolver dicha tarea.

Este problema se intensifica con la adopción de **sistemas agénticos**, en los cuales un LLM no opera de forma aislada sino que interactúa con otros agentes y herramientas como bases de datos, APIs, sistemas de archivos y servicios web estas siendo para completar tareas complejas del mundo real. El atributo dominante para esta interacción es el *tool calling*: el modelo recibe las definiciones (schemas) de las herramientas disponibles dentro de su contexto, selecciona cuál invocar, genera los parámetros apropiados para dicha herramienta, recibe el resultado, y continúa su razonamiento. Este proceso consume tokens: las descripciones de herramientas, los parámetros serializados, las respuestas intermedias de las herramientas y el historial acumulado de la conversación compiten por espacio dentro de la ventana de contexto.

En 2024, Anthropic introdujo el **Model Context Protocol** (MCP), un protocolo abierto que estandariza la comunicación entre LLMs y servidores de herramientas (Anthropic, 2024b). MCP define una interfaz cliente-servidor basada en JSON-RPC que permite a los modelos descubrir e invocar herra-

mientas de manera uniforme, independientemente de su implementación subyacente (Model Context Protocol, 2024). La adopción del protocolo ha crecido rápidamente en la industria, estableciéndose como un estándar de facto para la integración de herramientas en sistemas basados en LLMs.

Dentro del ecosistema MCP, han emergido varias maneras para la ejecución de tareas que requieren herramientas, en este caso se van a ver las siguientes:

1. **Llamadas directas a MCP**: el modelo invoca herramientas individuales a través de MCP, una por una, acumulando resultados en su contexto a lo largo de múltiples turnos de interacción.

2. **Ejecución de código**: el modelo genera un programa que orquesta múltiples llamadas a herramientas dentro de un entorno de ejecución aislado (*sandbox*), retornando únicamente el resultado final al contexto del modelo (Anthropic, 2024a).

Ambos esquemas resuelven el mismo problema funcional el cual es conectar un LLM con herramientas externas pero difieren radicalmente en cómo utilizan la ventana de contexto. Las llamadas directas a MCP cargan todas las definiciones de herramientas y resultados intermedios en el contexto del modelo en cada llamada; la ejecución de código delega la orquestación a un programa externo, reduciendo potencialmente el número de tokens consumidos a cambio de un overhead inicial de generación de código y ejecución en sandbox.

Esta distinción tiene implicaciones directas en las tres dimensiones que determinan la eficiencia de un sistema agéntico en producción: el **consumo de tokens** (costo económico), la **latencia** (experiencia del usuario) y la **precisión** (confiabilidad del sistema). No obstante, hasta la fecha no existe un estudio claro que cuantifique estos trade-offs de manera controlada y reproducible en sistemas agenticos. El presente trabajo busca llenar este vacío.

1.2. Planteamiento del Problema

El Model Context Protocol ha logrado estandarizar la manera en que los LLMs interactúan con herramientas externas, proporcionando un marco uniforme para el descubrimiento, invocación y manejo de resultados de herramientas. Sin embargo, a pesar de sus ventajas, el mecanismo convencional de *tool calling* sobre MCP presenta limitaciones estructurales que comprometen

su escalabilidad en terminos de costo y eficiencia en aplicaciones de producción (Anthropic, 2024a).

El primer problema es la **carga estática de definiciones de herramientas** (*static tool loading*). En el paradigma convencional de tool calling, todas las definiciones de herramientas disponibles incluyendo nombres, descripciones, parámetros y schemas deben cargarse en la ventana de contexto del modelo *antes* de que este procese el *prompt* del usuario. Esto significa que, en escenarios donde un agente está conectado a múltiples servidores MCP con decenas o cientos de herramientas, el modelo debe procesar cientos de miles de tokens de definiciones antes de siquiera leer la tarea por hacer Anthropic (2024a).Se señala que los agentes desperdician contexto significativo en descripciones que en su mayoría no serán utilizadas para la tarea actual.

El segundo problema es el **tránsito obligatorio de resultados intermedios por el contexto del modelo**. En una secuencia de llamadas a herramientas donde el resultado de una alimenta la entrada de la siguiente, cada resultado intermedio debe pasar por la ventana de contexto del modelo para ser interpretado y reformulado como entrada de la siguiente llamada. Esto implica que los mismos datos son procesados múltiples veces. Anthropic (2024a) ilustran este problema con un ejemplo concreto: en un flujo de recuperación y actualización de documentos, la transcripción completa de una llamada fluye por el contexto dos veces; esto significa procesar tokens adicionales innecesarios. En el caso de documentos extensos o estructuras de datos complejas, este patrón puede incluso exceder los límites de la ventana de contexto, haciendo la tarea irrealizable.

El tercer problema es la **propensión a errores en la manipulación de datos entre llamadas**. Cuando un modelo debe copiar datos de la respuesta de una herramienta hacia los parámetros de la siguiente, la probabilidad de errores de transcripción aumenta proporcionalmente con el volumen y la complejidad de los datos. Anthropic (2024a) advierten que, con documentos extensos o estructuras de datos complejas, los modelos son más propensos a cometer errores al copiar datos entre llamadas a herramientas.

Estos tres problemas, carga estática, tránsito redundante de datos y errores de transcripción comparten una raíz común: el **la armonía entre la orquestación de herramientas y la ventana de contexto del modelo**. En el paradigma convencional, el contexto del modelo cumple una doble función: es simultáneamente el espacio de razonamiento del LLM y el canal de comunica-

ción entre herramientas. Esta doble función genera una tensión fundamental entre la cantidad de herramientas disponibles y la eficiencia con que se utiliza el contexto.

Anthropic propone como alternativa la ejecución de código, donde el modelo genera programas que orquestan las llamadas a herramientas en un entorno externo, descubriendo y cargando definiciones bajo demanda en lugar de estáticamente. Este enfoque reporta reducciones de hasta 98.7 % en consumo de tokens de 150,000 a 2,000 tokens en escenarios con grandes conjuntos de herramientas. Sin embargo, esta cifra proviene de un caso de uso específico y no constituye una evaluación sistemática a través de diferentes tipos de tareas, niveles de complejidad y sistemas agenticos.

La pregunta central de esta investigación es, por tanto: **¿bajo qué condiciones la ejecución de código a través de MCP supera a las llamadas directas en términos de consumo de tokens, latencia y precisión en sistemas agenticos?**

1.3. Motivación

La resolución de los problemas descritos en la sección anterior no es un ejercicio puramente teórico. Las limitaciones del tool calling convencional sobre MCP tienen consecuencias tangibles que justifican una investigación sistemática desde múltiples perspectivas.

1.3.1. Impacto económico

En aplicaciones de producción, cada token procesado por un LLM tiene un costo monetario directo. Los proveedores de modelos —OpenAI, Anthropic, Google, Amazon— facturan por millón de tokens de entrada y de salida, con precios que varían según el modelo pero que en todos los casos escalan linealmente con el consumo. Cuando un agente conectado a múltiples servidores MCP carga estáticamente 150,000 tokens de definiciones de herramientas en cada interacción, ese costo se repite en *cada solicitud del usuario*, independientemente de cuántas herramientas se utilicen realmente. A escala —miles de usuarios, millones de solicitudes diarias— la diferencia entre una arquitectura que consume 150,000 tokens y una que consume 2,000 tokens por interacción puede representar órdenes de magnitud en costos operativos. Sin embargo, no existe una cuantificación rigurosa de estos ahorros a través de diferentes ti-

pos de tareas y configuraciones, lo cual impide que las organizaciones tomen decisiones informadas sobre su arquitectura.

1.3.2. Ausencia de guías para desarrolladores

El ecosistema de desarrollo de sistemas agénticos carece de criterios empíricos para elegir entre llamadas directas y ejecución de código. Un desarrollador que diseña un sistema basado en MCP enfrenta una decisión arquitectónica fundamental al inicio del proyecto: ¿debe su agente llamar herramientas directamente, o debe generar código que las orqueste? Esta decisión tiene implicaciones profundas en el diseño del sistema, la infraestructura necesaria (sandbox de ejecución, seguridad, monitoreo) y el comportamiento en producción. Actualmente, esta elección se basa en intuición, anécdotas o en las recomendaciones de un único artículo técnico (Anthropic, 2024a), sin un marco cuantitativo que relacione las características de la tarea con el modo de ejecución óptimo.

1.3.3. Crecimiento acelerado del ecosistema MCP

Desde su introducción en 2024, el Model Context Protocol ha experimentado una adopción acelerada. Empresas como Anthropic, Amazon Web Services, Microsoft y múltiples startups han construido servidores MCP para sus plataformas, y la comunidad open-source ha contribuido cientos de implementaciones que cubren desde bases de datos hasta APIs de servicios en la nube (Anthropic, 2024b). Esta proliferación de servidores y herramientas agrava directamente el problema de la carga estática: conforme un agente se conecta a más servidores, el volumen de definiciones de herramientas en su contexto crece proporcionalmente. Lo que hoy es un problema manejable con un puñado de herramientas se convertirá en un cuello de botella crítico conforme el ecosistema madure y los agentes necesiten acceso a decenas de servidores simultáneamente.

1.3.4. Vacío en la literatura académica

A pesar de la rápida adopción industrial del protocolo MCP y de los patrones de ejecución de código, la literatura académica no ha abordado de manera sistemática la comparación entre estos dos paradigmas de ejecución. Existen benchmarks para evaluar la capacidad general de tool-use de los LLMs, así co-

mo evaluaciones de sistemas multi-agente en tareas específicas, pero ninguno se enfoca en el eje particular de *cómo* se ejecutan las herramientas y su impacto en eficiencia. Esta ausencia de evidencia empírica deja un vacío que este trabajo busca llenar mediante un diseño experimental controlado, reproducible y extensible por la comunidad.

1.4. Objetivos

1.4.1. Objetivo General

1.4.2. Objetivos Específicos

1.5. Hipótesis

1.6. Alcance y Limitaciones

2. Revisión de Literatura

2.1. Protocolo MCP (Model Context Protocol)

2.1.1. Origen y motivación

A medida que los LLMs se integraron en aplicaciones de producción, surgió un problema recurrente: cada fuente de datos o herramienta externa requería una implementación personalizada para conectarse con el modelo. Un agente que necesitaba acceder a una base de datos, un sistema de archivos y una API de terceros debía implementar tres conectores distintos, cada uno con su propia lógica de autenticación, serialización y manejo de errores. Esta fragmentación hacía que la construcción de sistemas verdaderamente conectados fuera difícil de escalar (Anthropic, 2024b).

En noviembre de 2024, Anthropic introdujo el Model Context Protocol (MCP) como un estándar abierto para resolver este problema. MCP proporciona un protocolo uniforme para conectar aplicaciones de inteligencia artificial con los sistemas donde residen los datos, estableciendo conexiones bidireccionales y seguras entre fuentes de datos y aplicaciones basadas en LLMs (Anthropic, 2024b). La analogía oficial del protocolo lo compara con un puerto USB-C: así como USB-C estandarizó la conexión entre dispositivos electrónicos, MCP estandariza la conexión entre aplicaciones de IA y sistemas externos (Model Context Protocol, 2024).

2.1.2. Arquitectura cliente-servidor

MCP sigue una arquitectura cliente-servidor con tres participantes principales (Model Context Protocol, 2024):

- **MCP Host**: la aplicación de IA que coordina y gestiona uno o múltiples clientes MCP. Ejemplos incluyen Claude Desktop, Visual Studio Code con Copilot, Cursor y Claude Code.
- **MCP Client**: un componente dentro del host que mantiene una conexión dedicada con un servidor MCP específico. El host instancia un cliente por cada servidor al que se conecta.
- **MCP Server**: un programa que expone datos, herramientas o plantillas de interacción a los clientes MCP. Los servidores pueden ejecutarse

localmente (en la misma máquina que el host) o de forma remota (en un servidor web).

Esta separación permite que un mismo host se conecte simultáneamente a múltiples servidores —por ejemplo, un servidor de sistema de archivos local, un servidor de base de datos y un servidor remoto de Sentry— manteniendo conexiones independientes y aisladas para cada uno.

2.1.3. Capas del protocolo

El protocolo se estructura en dos capas conceptuales (Model Context Protocol, 2024):

Capa de datos. Implementa un protocolo de intercambio basado en JSON-RPC 2.0 (JSON-RPC Working Group, 2010) que define la estructura y semántica de los mensajes. Esta capa incluye:

- **Gestión del ciclo de vida**: manejo de la inicialización de conexión, negociación de capacidades y terminación. Durante la inicialización, cliente y servidor intercambian sus capacidades soportadas (herramientas, recursos, prompts) y versión del protocolo mediante un handshake formal.
- **Primitivas de servidor**: los tres tipos fundamentales de contexto que un servidor puede exponer:
  - *Tools* (herramientas): funciones ejecutables que la aplicación de IA puede invocar para realizar acciones —operaciones de archivos, consultas a bases de datos, llamadas a APIs.
  - *Resources* (recursos): fuentes de datos que proveen información contextual —contenido de archivos, registros de base de datos, respuestas de APIs.
  - *Prompts* (plantillas): templates reutilizables que estructuran las interacciones con el modelo.
- **Primitivas de cliente**: capacidades que el cliente expone al servidor, incluyendo *sampling* (solicitar completions al LLM del host), *elicitation* (solicitar información adicional al usuario) y *logging* (enviar mensajes de depuración).

- **Notificaciones**: mensajes en tiempo real que permiten a los servidores informar a los clientes sobre cambios sin necesidad de polling —por ejemplo, cuando la lista de herramientas disponibles cambia dinámicamente.

Capa de transporte. Define los mecanismos de comunicación entre clientes y servidores. MCP soporta dos mecanismos de transporte:

- **Stdio**: utiliza flujos de entrada/salida estándar para comunicación directa entre procesos locales en la misma máquina. Ofrece rendimiento óptimo sin overhead de red y es el mecanismo típico para servidores locales.

- **Streamable HTTP**: utiliza HTTP POST para mensajes del cliente al servidor con Server-Sent Events (SSE) opcionales para streaming. Habilita la comunicación con servidores remotos y soporta métodos estándar de autenticación HTTP.

La capa de transporte abstrae los detalles de comunicación, permitiendo que el mismo formato de mensajes JSON-RPC 2.0 funcione de manera idéntica sobre cualquier mecanismo de transporte.

2.1.4. Flujo de interacción

Una interacción típica entre un cliente y un servidor MCP sigue una secuencia bien definida (Model Context Protocol, 2024):

1. **Inicialización**: el cliente envía una solicitud `initialize` con su versión del protocolo y capacidades soportadas. El servidor responde con las suyas. Ambas partes negocian qué primitivas y funcionalidades estarán disponibles durante la sesión.

2. **Descubrimiento de herramientas**: el cliente invoca `tools/list` para obtener la lista completa de herramientas disponibles, incluyendo nombre, descripción y schema de entrada (`inputSchema`) definido como JSON Schema para cada una.

3. **Ejecución de herramientas**: cuando el LLM del host decide utilizar una herramienta, el cliente envía una solicitud `tools/call` con el nombre de la herramienta y sus argumentos. El servidor ejecuta la operación y

retorna el resultado como un arreglo de objetos de contenido (texto, imágenes, recursos).

4. **Actualización dinámica**: si la lista de herramientas cambia durante la sesión, el servidor envía una notificación `notifications/tools/list_changed` y el cliente solicita la lista actualizada.

2.1.5. Adopción del ecosistema

Desde su lanzamiento, MCP ha experimentado una adopción amplia tanto en la industria como en la comunidad open-source. En el lanzamiento inicial, Anthropic publicó la especificación abierta, SDKs para Python y TypeScript, y servidores de referencia para Google Drive, Slack, GitHub, Git, Postgres y Puppeteer (Anthropic, 2024b). Empresas como Block y Apollo integraron el protocolo en sus operaciones, mientras que plataformas de desarrollo como Zed, Replit, Codeium y Sourcegraph lo adoptaron para mejorar las capacidades contextuales de sus herramientas de IA.

A la fecha, MCP cuenta con soporte en aplicaciones como Claude Desktop, ChatGPT, Visual Studio Code, Cursor y numerosos clientes adicionales (Model Context Protocol, 2024), consolidándose como el estándar de facto para la integración de herramientas en sistemas basados en LLMs.

2.2. Ejecución de Código en LLMs

La capacidad de los LLMs para generar código ejecutable ha dado lugar a un paradigma de interacción fundamentalmente distinto al tool calling convencional. En lugar de que el modelo invoque herramientas una por una a través de una interfaz estructurada, el modelo genera un programa completo que orquesta múltiples operaciones dentro de un entorno de ejecución aislado. Esta sección revisa los principales patrones y plataformas que han emergido para habilitar la ejecución segura de código generado por LLMs.

2.2.1. El patrón de Anthropic: herramientas como código

Anthropic (2024a) proponen una reestructuración fundamental en la forma en que los agentes interactúan con servidores MCP. En lugar de presentar las herramientas como funciones invocables a través del protocolo de tool calling, las reorganizan como un **sistema de archivos navegable** donde cada

herramienta es un módulo de código con una interfaz tipada:

Listing 1: Estructura de herramientas como archivos

```
servers/
  google-drive/
    getDocument.ts
    listFiles.ts
    index.ts
  salesforce/
    updateRecord.ts
    queryRecords.ts
    index.ts
```

Cada módulo exporta una función con tipos de entrada y salida explícitos. El agente, en lugar de recibir todas las definiciones de herramientas en su contexto de manera estática, **navega el sistema de archivos bajo demanda**: lista los servidores disponibles, lee las definiciones de las herramientas que necesita para la tarea actual, y genera código que las orquesta directamente.

Este patrón invierte el flujo de información. En el tool calling convencional, el contexto del modelo funciona como intermediario: cada resultado intermedio pasa por la ventana de contexto para ser interpretado y reformulado como entrada de la siguiente herramienta. En la ejecución de código, los datos fluyen programáticamente entre funciones dentro del entorno de ejecución, y únicamente el resultado final regresa al contexto del modelo. Como ilustran Anthropic (2024a), una tarea que requiere recuperar un documento de Google Drive y actualizar un registro en Salesforce se reduce a un programa de pocas líneas donde los datos intermedios nunca pasan por el modelo:

Listing 2: Orquestación directa mediante código

```
transcript = gdrive.get_document(document_id="abc123")
salesforce.update_record(
    object_type="SalesMeeting",
    record_id="00Q5f000001abc",
    data={"Notes": transcript.content}
)
```

El resultado reportado es una reducción de 150,000 a 2,000 tokens —un ahorro del 98.7 %— en escenarios donde el agente está conectado a múltiples servidores con grandes conjuntos de herramientas (Anthropic, 2024a).

2.2.2. AWS Strands: code interpreter como herramienta

Amazon Web Services adoptó un enfoque complementario a través de su framework Strands Agents (AWS Strands Agents, 2025). En lugar de reestructurar la arquitectura de los servidores MCP, Strands integra un *code interpreter* como una herramienta más dentro del toolkit del agente. El agente puede decidir, durante la ejecución de una tarea, si invocar herramientas directamente o generar código Python que será ejecutado en un entorno administrado por Amazon Bedrock AgentCore.

Este patrón es significativo porque no requiere modificar la infraestructura de servidores MCP existente: el code interpreter actúa como una capa intermedia que permite al agente escribir código que a su vez invoca herramientas convencionales. La decisión de cuándo usar ejecución de código versus llamadas directas queda delegada al propio modelo, basándose en el contexto de la tarea.

2.2.3. E2B: microVMs para ejecución aislada

E2B (E2B, 2024) ofrece una plataforma de código abierto especializada en proveer entornos de ejecución seguros para agentes de IA. A diferencia de los enfoques basados en contenedores Docker, E2B utiliza **microVMs basadas en Firecracker** —la misma tecnología desarrollada por Amazon para AWS Lambda— que proporcionan aislamiento a nivel de kernel en lugar de compartir el kernel del host.

Las características clave de E2B incluyen:

- **Inicio rápido**: cada sandbox inicia en menos de 200ms, habilitando la creación de entornos efímeros por tarea.
- **Aislamiento fuerte**: cada microVM ejecuta su propio microkernel, proporcionando aislamiento superior al de contenedores para código no confiable generado por IA.
- **Entorno completo**: cada sandbox es un sistema Linux completo con terminal, sistema de archivos, instalación de paquetes y persistencia de sesión de hasta 24 horas.
- **Agnóstico al modelo**: compatible con OpenAI, Anthropic, Mistral, Llama y modelos personalizados.

E2B representa el extremo del espectro de aislamiento: mientras que Docker ofrece aislamiento a nivel de proceso compartiendo el kernel del host, y Firecracker ofrece aislamiento a nivel de VM con overhead mínimo, cada plataforma presenta un trade-off distinto entre seguridad y rendimiento.

2.2.4. Cloudflare Workers: WebAssembly como sandbox

Cloudflare Workers ofrece un enfoque radicalmente diferente mediante el uso de **V8 isolates y WebAssembly** como mecanismo de aislamiento (Cloudflare, 2024). En este modelo, el código Python se ejecuta a través de Pyodide —un port de CPython a WebAssembly— dentro de isolates de V8, el mismo motor de JavaScript que utiliza Chrome.

Este enfoque tiene implicaciones relevantes para la ejecución de código de agentes:

- El modelo de ejecución WebAssembly proporciona sandboxing inherente: el código Python ejecuta dentro de un módulo WebAssembly aislado del runtime del host.
- Todas las operaciones fuera del cómputo puro deben ser explícitamente importadas a través de bindings del runtime.
- El overhead de inicio se mitiga mediante snapshots de memoria que capturan dependencias pre-cargadas en tiempo de despliegue.

A diferencia de Docker y Firecracker, que operan a nivel de sistema operativo, WebAssembly opera a nivel de instrucciones, ofreciendo aislamiento por construcción del formato binario mismo.

2.2.5. Comparación de enfoques

Los cuatro enfoques revisados comparten el objetivo de habilitar la ejecución segura de código generado por LLMs, pero difieren significativamente en su arquitectura, nivel de aislamiento y trade-offs:

Característica	Docker	Firecracker	WebAssembly	Strands
Aislamiento	Proceso	Kernel	Instrucción	Administrado
Inicio	∼1s	<200ms	<50ms	Variable
Overhead	Medio	Bajo	Mínimo	Opaco
Entorno	Linux completo	Linux completo	Limitado	Administrado
Código abierto	Sí	Sí	Sí	Parcial

Tabla 1: Comparación de plataformas de ejecución de código para agentes de IA

El presente trabajo utiliza Docker como entorno de ejecución por su balance entre aislamiento, flexibilidad de entorno (sistema Linux completo con bibliotecas arbitrarias) y facilidad de implementación en un contexto de investigación. La elección de sandbox no es el foco de la investigación, sino un componente de infraestructura controlado para evaluar el paradigma de ejecución de código en sí mismo.

2.3. Sistemas Multi-Agente

A medida que las tareas asignadas a agentes basados en LLMs crecen en complejidad, un único agente con acceso a todas las herramientas disponibles enfrenta limitaciones fundamentales: la ventana de contexto se satura con definiciones de herramientas, la especialización se diluye, y la coordinación de subtareas heterogéneas recae enteramente en las capacidades de razonamiento del modelo. Los sistemas multi-agente abordan estas limitaciones mediante la descomposición de tareas complejas en subtareas asignadas a agentes especializados, cada uno con un conjunto reducido de herramientas y un rol claramente definido.

2.3.1. Topologías de orquestación

La literatura y los frameworks existentes han convergido en tres topologías principales para organizar sistemas multi-agente:

Orquestador centralizado. Un agente coordinador recibe la tarea principal, la descompone en subtareas y las delega a agentes especializados. El orquestador mantiene el estado global de la tarea y sintetiza los resultados parciales en una respuesta coherente. Esta topología ofrece control explícito sobre el flujo de ejecución, pero introduce un cuello de botella: todo el contexto

intermedio debe pasar por el orquestador, incrementando el consumo de tokens. Es el patrón utilizado en las configuraciones A3 y A4 de esta tesis, donde un agente principal delega a sub-agentes especializados por servidor MCP.

Enjambre (swarm). En esta topología, los agentes se transfieren el control mutuamente sin un coordinador central. Cada agente decide, basándose en su salida, a qué agente transferir la tarea siguiente. OpenAI popularizó este patrón con su framework experimental Swarm, donde las funciones de handoff determinan dinámicamente el flujo de ejecución. La ventaja es la flexibilidad y la ausencia de un punto único de fallo; la desventaja es la dificultad de garantizar convergencia y la opacidad del flujo de ejecución.

Pipeline/DAG. Los agentes se organizan en un grafo acíclico dirigido (DAG) con dependencias explícitas entre nodos. Cada nodo representa un agente o una transformación, y las aristas definen el flujo de datos. Esta topología es natural para workflows deterministas donde el orden de ejecución es predecible — por ejemplo, extracción → transformación → carga. LangGraph (LangChain, 2024) implementa esta topología como su abstracción principal.

Los trade-offs entre estas topologías se resumen en la capacidad de coordinación versus la complejidad del sistema. El orquestador centralizado maximiza el control a costa de concentrar el consumo de tokens; el enjambre minimiza la coordinación explícita pero dificulta la depuración; el pipeline/DAG ofrece predictibilidad pero limita la flexibilidad ante tareas no anticipadas.

2.3.2. Frameworks existentes

Múltiples frameworks han emergido para facilitar la construcción de sistemas multi-agente. Esta sección revisa los más relevantes para el contexto de esta investigación.

AutoGen. Desarrollado por Microsoft Research (Wu et al., 2023), AutoGen modela la colaboración multi-agente como una *conversación* entre agentes. Su abstracción central es el `GroupChat`, donde múltiples agentes interactúan en turnos definidos por un selector automático o por reglas explícitas. AutoGen soporta agentes con capacidad de ejecución de código integrada y permite la intervención humana en el flujo conversacional. Su paradigma conversacional

es particularmente adecuado para tareas que requieren deliberación iterativa entre agentes con roles distintos.

CrewAI. CrewAI (Moura, 2024) adopta una metáfora de *equipo de trabajo*: cada agente tiene un rol, un objetivo y un conjunto de herramientas. Las tareas se definen con dependencias explícitas y se asignan a agentes específicos. CrewAI distingue entre ejecución secuencial (pipeline) y jerárquica (con un agente manager que coordina). Su principal contribución es la simplicidad de configuración para workflows orientados a roles.

LangGraph. LangGraph (LangChain, 2024), desarrollado por LangChain, modela workflows de agentes como grafos de estado donde los nodos representan funciones o agentes y las aristas definen transiciones condicionales. Su abstracción de *state* permite persistir y restaurar el estado completo de una ejecución, habilitando patrones como checkpointing, human-in-the-loop y ejecución distribuida. LangGraph es agnóstico al modelo y al proveedor de herramientas.

Strands SDK. AWS Strands Agents (Amazon Web Services, 2025) implementa un patrón denominado *agents-as-tools*: cada sub-agente se encapsula como una herramienta invocable por un agente orquestador. Este patrón permite componer sistemas multi-agente utilizando la misma interfaz de tool calling que se usa para herramientas convencionales, sin requerir abstracciones adicionales de orquestación. Strands es el framework utilizado en la implementación de esta tesis, por lo que su patrón de composición se detalla en el Capítulo 4.

Característica	**AutoGen**	**CrewAI**	**LangGraph**	**Strands**
Paradigma	Conversación	Roles/equipo	Grafo de estado	Agente-herramienta
Orquestación	GroupChat	Secuencial/jerárq.	DAG condicional	Centralizada
Integración MCP	Indirecta	Indirecta	Indirecta	Nativa
Code execution	Integrado	Mediante tools	Mediante nodos	Code interpreter

Tabla 2: Comparación de frameworks multi-agente

2.3.3. Patrones de comunicación

Independientemente del framework utilizado, los sistemas multi-agente emplean patrones de comunicación recurrentes:

Agentes como herramientas. El sub-agente se expone como una herramienta callable con un schema de entrada y salida definido. El orquestador invoca al sub-agente de la misma forma que invocaría cualquier herramienta MCP, y recibe el resultado como texto estructurado. Este es el patrón implementado por Strands SDK y utilizado en las configuraciones A3 y A4 de esta investigación. Su principal ventaja es la composabilidad: los sub-agentes se pueden anidar arbitrariamente sin cambiar el protocolo de comunicación.

Paso de mensajes. Los agentes intercambian mensajes estructurados a través de un canal compartido o mediante handoffs explícitos. Cada agente mantiene su propio historial de conversación y contexto. AutoGen implementa este patrón mediante su abstracción de chat grupal, donde los mensajes son visibles para todos los participantes del grupo.

Contexto compartido. Un broker de contexto centralizado mantiene un estado compartido al que todos los agentes pueden leer y escribir. Este patrón reduce la duplicación de información pero introduce riesgos de consistencia cuando múltiples agentes modifican el estado concurrentemente.

Impacto en el consumo de tokens. La elección del patrón de comunicación tiene implicaciones directas en el consumo de tokens. En el patrón de agentes como herramientas, cada invocación de sub-agente genera un ciclo completo de LLM (prompt del sub-agente + respuesta), multiplicando el consumo de tokens por el número de delegaciones. En el paso de mensajes, cada mensaje en el historial compartido incrementa el contexto de todos los participantes. Estas implicaciones motivan la hipótesis de esta tesis: la ejecución de código puede reducir significativamente el consumo de tokens al eliminar ciclos intermedios de LLM en cadenas de herramientas.

2.4. Benchmarks Existentes

La evaluación sistemática de las capacidades de los LLMs para utilizar herramientas y completar tareas complejas ha motivado el desarrollo de múltiples benchmarks especializados. Esta sección revisa los más relevantes, organizados en dos categorías: benchmarks de tool-use (que evalúan la capacidad de invocar herramientas correctamente) y benchmarks de agentes (que evalúan la capacidad de completar tareas end-to-end). La sección concluye identificando el vacío que esta investigación busca llenar.

2.4.1. Benchmarks de tool-use

Los benchmarks de tool-use evalúan la capacidad de un LLM para seleccionar herramientas apropiadas, generar parámetros correctos y encadenar múltiples llamadas cuando es necesario.

ToolBench. Qin et al. (2023) presentan un benchmark a gran escala con más de 16,000 APIs del mundo real organizadas en categorías funcionales. ToolBench evalúa no solo la selección de herramientas individuales, sino la capacidad del modelo para resolver tareas que requieren múltiples llamadas a APIs heterogéneas. Los autores proponen un método de planificación basado en árboles de decisión (DFSDT) que mejora significativamente el rendimiento en cadenas de herramientas complejas. La principal contribución de ToolBench es demostrar que la escala del espacio de herramientas —cientos o miles de APIs disponibles— es un factor crítico en el rendimiento de los modelos.

API-Bank. Li et al. (2023) construyen un benchmark para evaluar LLMs aumentados con herramientas a través de tres niveles de dificultad: Nivel 1 evalúa la invocación correcta de una API individual; Nivel 2 requiere recuperar la API apropiada de un conjunto amplio; Nivel 3 demanda planificación y ejecución de múltiples APIs con dependencias entre ellas. API-Bank proporciona un entorno de ejecución donde las llamadas a APIs se ejecutan realmente, permitiendo evaluar no solo la generación de parámetros sino la corrección funcional de las respuestas.

Berkeley Function Calling Leaderboard (BFCL). El BFCL (Gorilla LLM Team, 2024) estandariza la evaluación de function calling a través de

múltiples categorías: llamadas simples, llamadas múltiples, llamadas paralelas, selección de funciones relevantes de un conjunto amplio, y detección de consultas que no requieren ninguna herramienta. Su contribución principal es la **estandarización**: al proveer un leaderboard público con métricas uniformes, permite comparar directamente la capacidad de function calling entre modelos de diferentes proveedores bajo condiciones idénticas.

2.4.2. Benchmarks de agentes

A diferencia de los benchmarks de tool-use, que evalúan interacciones aisladas con herramientas, los benchmarks de agentes evalúan la capacidad de completar tareas complejas que requieren múltiples pasos de razonamiento, planificación y ejecución.

SWE-Bench. Jimenez et al. (2024) presentan un benchmark que evalúa la capacidad de agentes para resolver issues reales de repositorios de código abierto en GitHub. Cada instancia consiste en una descripción de un bug o feature request y el estado del repositorio correspondiente; el agente debe producir un patch que resuelva el issue y pase los tests del repositorio. SWE-Bench es particularmente relevante porque evalúa una cadena completa de capacidades: comprensión de código existente, razonamiento sobre el problema, generación de código y validación mediante tests. Los resultados reportados muestran que incluso los mejores agentes resuelven menos del 30 % de las instancias, evidenciando la dificultad de las tareas de ingeniería de software autónoma.

WebArena. Zhou et al. (2024) construyen un entorno realista de navegación web con sitios funcionales (e-commerce, foros, herramientas de gestión de contenido) donde los agentes deben completar tareas complejas mediante interacciones con la interfaz web. WebArena evalúa la capacidad del agente para planificar secuencias de acciones en un entorno dinámico donde el estado cambia con cada interacción, un escenario análogo al uso de herramientas MCP donde cada llamada puede alterar el estado del sistema.

GAIA. Mialon et al. (2024) proponen un benchmark para evaluar asistentes de IA generales en tareas que requieren razonamiento multi-paso, uso de herramientas web y manejo de archivos multimodales. Las tareas de GAIA están

diseñadas para ser fáciles de verificar (las respuestas son factuales y unívocas) pero difíciles de resolver, requiriendo que el agente combine múltiples capacidades. GAIA es relevante para esta investigación porque sus tareas implican frecuentemente cadenas de herramientas —exactamente el escenario donde la ejecución de código puede ofrecer ventajas sobre el tool calling convencional.

2.4.3. El vacío identificado

Los benchmarks revisados comparten una característica en común: todos evalúan *qué* logra el agente (correctitud de la respuesta, resolución del issue, completitud de la tarea), pero ninguno evalúa *cómo* el agente ejecuta las herramientas. Específicamente, se identifican tres vacíos en la literatura:

1. **Modo de ejecución como variable**: ningún benchmark existente compara el rendimiento de un mismo agente utilizando tool calling convencional versus ejecución de código como modos alternativos de interacción con herramientas.

2. **Eficiencia de tokens**: los benchmarks existentes reportan métricas de éxito/fracaso, pero no miden sistemáticamente el consumo de tokens como función del modo de ejecución. Dado que el costo operativo de los agentes es directamente proporcional al consumo de tokens, esta métrica es crítica para la viabilidad de despliegues en producción.

3. **Complejidad de herramientas como variable**: no existe un benchmark que varíe sistemáticamente la complejidad del servidor de herramientas (número de herramientas, grado de concatenación requerido) para evaluar cómo escala cada modo de ejecución.

Esta tesis aborda estos vacíos mediante un diseño experimental factorial 2×2 (modo de ejecución $\times$ número de agentes) evaluado sobre servidores MCP de complejidad creciente, midiendo simultáneamente correctitud, consumo de tokens y latencia. El diseño experimental se detalla en el Capítulo 3.

2.5. Sandboxing y Seguridad

La Sección 2.2 comparó las plataformas disponibles para ejecutar código generado por LLMs. Esta sección complementa esa discusión enfocándose en el **modelo de amenazas** específico de la ejecución de código de IA y las estrategias de mitigación implementadas en esta investigación.

2.5.1. Modelo de amenazas

El código generado por un LLM es inherentemente no confiable. A diferencia del código escrito por desarrolladores humanos —que pasa por procesos de revisión, testing y control de versiones—, el código generado por un modelo puede contener errores sutiles producto de alucinaciones, efectos no intencionados de prompt injection, o simplemente lógica incorrecta que el modelo no tiene capacidad de verificar antes de la ejecución. Esto establece un modelo de amenazas con cuatro categorías principales:

1. **Escape del sandbox**: código que explota vulnerabilidades del entorno de ejecución para obtener acceso al sistema host. En el caso de Docker, esto incluye exploits del kernel compartido, escape mediante `/proc` o `/sys`, y abuso de capabilities del contenedor.

2. **Exfiltración de datos**: código que utiliza acceso a red para enviar datos sensibles a servidores externos. Un LLM que genera código con llamadas HTTP ocultas podría exfiltrar datos del volumen montado o información del entorno de ejecución.

3. **Denegación de servicio**: código que consume recursos del host de manera desproporcionada —bucles infinitos, asignaciones masivas de memoria, fork bombs— degradando o interrumpiendo el servicio para otros procesos en la máquina.

4. **Acceso al sistema host**: código que intenta acceder al sistema de archivos del host fuera de los volúmenes explícitamente montados, escalar privilegios, o manipular procesos del host a través de interfaces compartidas.

Es importante notar que estas amenazas no requieren intención maliciosa del usuario. Un LLM puede generar código peligroso como efecto secundario de instrucciones benignas —por ejemplo, un modelo que intenta "optimizar" una consulta podría generar código que itera indefinidamente, o un modelo que intenta "explorar" el sistema podría generar código que accede a rutas sensibles.

2.5.2. Estrategias de mitigación en Docker

La implementación de esta tesis utiliza Docker como entorno de ejecución con una configuración de seguridad que mitiga cada categoría de amenaza identificada. Los contenedores se ejecutan con los siguientes flags de seguridad:

Listing 3: Flags de seguridad Docker utilizados en esta investigación

```
docker run \
  --network none \
  --memory 512m \
  --cpus 1 \
  --read-only \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  -v ./data:/data:rw
```

Cada flag aborda amenazas específicas:

- `-network none`: elimina completamente el acceso a red del contenedor, mitigando la exfiltración de datos. El código ejecutado no puede establecer conexiones salientes ni recibir conexiones entrantes.
- `-memory 512m`: establece un límite máximo de memoria de 512 megabytes. Si el proceso excede este límite, el kernel del host termina el contenedor mediante OOM killer, mitigando la denegación de servicio por consumo de memoria.
- `-cpus 1`: limita el contenedor a un núcleo de CPU, previniendo que un proceso intensivo monopolice los recursos de cómputo del host.
- `-read-only`: monta el sistema de archivos raíz del contenedor como solo lectura. El código ejecutado no puede modificar binarios del sistema, instalar software adicional ni alterar la configuración del contenedor. Únicamente el directorio `/data`, montado explícitamente como lectura-escritura, es modificable.
- `-cap-drop ALL`: elimina todas las Linux capabilities del proceso del contenedor. Las capabilities son permisos granulares que el kernel otorga a procesos —como `NET_RAW` (sockets raw), `SYS_ADMIN` (operaciones de administración) o `MKNOD` (creación de dispositivos). Al eliminarlas todas, el proceso ejecuta con el mínimo privilegio posible.

- `-security-opt no-new-privileges`: previene que el proceso o sus hijos adquieran privilegios adicionales mediante mecanismos como `setuid`, `setgid` o ejecución de binarios con capabilities de archivo. Esto cierra vectores de escalamiento de privilegios dentro del contenedor.

Adicionalmente, los contenedores son **efímeros**: se crean para cada ejecución de código y se destruyen inmediatamente después. Esto garantiza que no existe persistencia de estado entre ejecuciones, eliminando la posibilidad de que código malicioso deje artefactos que afecten ejecuciones posteriores.

2.5.3. Alternativas de aislamiento

Aunque Docker proporciona un nivel de aislamiento adecuado para esta investigación, existen alternativas con garantías de seguridad más fuertes:

gVisor. Desarrollado por Google (Google, 2024), gVisor implementa un kernel de aplicación en espacio de usuario que intercepta todas las llamadas al sistema (syscalls) del contenedor. A diferencia de Docker estándar, donde los contenedores comparten el kernel del host y las syscalls se ejecutan directamente, gVisor actúa como un proxy que reimplementa un subconjunto del kernel Linux en Go. Esto reduce drásticamente la superficie de ataque: un exploit de kernel en un contenedor con gVisor afecta únicamente al kernel de aplicación de gVisor, no al kernel del host. El trade-off es un overhead de rendimiento en operaciones intensivas de syscalls.

Firecracker. Como se describió en la Sección 2.2, Firecracker (E2B, 2024) proporciona aislamiento a nivel de microVM, ejecutando un kernel independiente para cada sandbox. Esto elimina por completo el vector de escape de kernel compartido, ya que cada ejecución opera sobre su propio kernel. El costo es mayor consumo de recursos por instancia comparado con contenedores.

WebAssembly. El sandboxing a nivel de instrucción de WebAssembly, descrito en la Sección 2.2, ofrece aislamiento por construcción: el código ejecuta dentro de un módulo binario que no tiene acceso directo al sistema operativo. Sin embargo, el soporte limitado de bibliotecas Python (a través de Pyodide) lo hace menos adecuado para escenarios que requieren dependencias científicas o de análisis de datos.

El trade-off general entre estas alternativas es claro: **mayor aislamiento implica mayor overhead**, lo que potencialmente impacta la latencia medida en los experimentos. La Tabla 1 en la Sección 2.2 cuantifica estas diferencias.

2.5.4. Implicaciones para la investigación

La configuración de seguridad tiene implicaciones directas para el diseño experimental de esta tesis:

- **Variable controlada**: las cuatro configuraciones experimentales (A1–A4) utilizan exactamente los mismos flags de seguridad de Docker. Esto asegura que cualquier diferencia observada en latencia o rendimiento se debe al modo de ejecución (tool calling vs. código) y no a diferencias en la configuración del sandbox.

- **Componente de la latencia medida**: el overhead de crear un contenedor Docker, ejecutar el código y destruir el contenedor es parte de la latencia total medida en las configuraciones A2 y A4 (ejecución de código). Este overhead es un costo inherente del paradigma de ejecución de código que los resultados deben reflejar de manera transparente.

- **Limitación del timeout**: cada ejecución tiene un timeout configurable (30 segundos por defecto) que actúa como una capa adicional de protección contra denegación de servicio y como una variable controlada del experimento.

3. Marco Teórico

3.1. Arquitectura MCP

3.2. Mecánica de Tool Calling

3.3. Ejecución de Código en Sandbox

3.4. Topologías Multi-Agente

3.5. Economía de Tokens

4. Diseño Experimental

4.1. Matriz Factorial 2×2

4.2. Suite de Tareas

4.3. Servidores MCP Utilizados

4.4. Métricas

4.5. Variables de Control

5. Metodología

5.1. Implementación del Framework

5.2. Sandbox Docker

5.3. Servidores MCP

5.4. Runner de Experimentos

5.5. Sistema de Evaluación

6. Resultados

6.1. Resultados por Arquitectura

6.2. Análisis de Tokens

6.3. Análisis de Latencia

6.4. Análisis de Precisión

6.5. Análisis Estadístico

7. Modelo de Decisión

7.1. Feature Engineering

7.2. Entrenamiento del Clasificador

7.3. Validación

7.4. Árbol de Decisión Interpretable

8. Discusión

8.1. Interpretación de Resultados

8.2. Comparación con Literatura

8.3. Análisis de Modos de Fallo

8.4. Implicaciones Prácticas

9. Conclusiones

9.1. Resumen de Hallazgos

9.2. Contribuciones

9.3. Trabajo Futuro

Referencias

Amazon Web Services. Strands Agents SDK: A model-driven approach to building AI agents, 2025. URL https://github.com/strands-agents/sdk-python. Accessed: 2026-03-11.

Anthropic. Code execution with mcp, 2024a. URL https://www.anthropic.com/engineering/code-execution-with-mcp. Accessed: 2026-03-09.

Anthropic. Introducing the model context protocol, 2024b. URL https://www.anthropic.com/news/model-context-protocol. Accessed: 2026-03-09.

Anthropic. Claude opus 4.6 model card, 2025. URL https://www.anthropic.com/claude. Accessed: 2026-03-09.

AWS Strands Agents. Strands agents tools, 2025. URL https://github.com/strands-agents/tools. Accessed: 2026-03-09.

Cloudflare. Python workers: Running python on cloudflare workers, 2024. URL https://blog.cloudflare.com/python-workers. Accessed: 2026-03-11.

E2B. E2b: Open-source secure sandboxes for ai agents, 2024. URL https://e2b.dev. Accessed: 2026-03-11.

Google. gVisor: Application kernel for containers, 2024. URL https://gvisor.dev. Accessed: 2026-03-11.

Google DeepMind. Gemini 3: A new generation of multimodal models, 2025. URL https://deepmind.google/technologies/gemini/. Accessed: 2026-03-09.

Gorilla LLM Team. Berkeley function calling leaderboard, 2024. URL https://gorilla.cs.berkeley.edu/leaderboard.html. Accessed: 2026-03-11.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

JSON-RPC Working Group. JSON-RPC 2.0 specification, 2010. URL https://www.jsonrpc.org/specification. Accessed: 2026-03-10.

LangChain. LangGraph: Build resilient language agents as graphs, 2024. URL https://github.com/langchain-ai/langgraph. Accessed: 2026-03-11.

Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. *arXiv preprint arXiv:2304.08244*, 2023.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2024.

Model Context Protocol. Mcp specification, 2024. URL https://modelcontextprotocol.io/. Accessed: 2026-03-09.

João Moura. CrewAI: Framework for orchestrating role-playing autonomous AI agents, 2024. URL https://github.com/crewAIInc/crewAI. Accessed: 2026-03-11.

OpenAI. GPT-5 technical report, 2025. URL https://openai.com/research/gpt-5. Accessed: 2026-03-09.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2024.

A. Apéndice

A.1. Código Fuente de Servidores MCP

A.2. Tablas Completas de Resultados

A.3. Esquemas de Servidores MCP

A.4. Prompts Utilizados